

TUGAS 1

Genetic Algorithm Optimization Program

GA-Based Feature Selection for Breast Cancer Classification

Mata Kuliah: Kecerdasan Komputasional dan Pembelajaran Mesin
(Computational Intelligence and Machine Learning)

Dosen: Oskar Natan, S.ST., M.Tr.T., Ph.D.

Disusun oleh:

Nama : **OSAMA YEHYA GHAZAL**

NIM : **26/591650/PPA/07466**

Program Studi : **MAGISTER ILMU KOMPUTER**

**PROGRAM STUDI MAGISTER ILMU KOMPUTER
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

Source code: <https://github.com/osamaygh/ga-feature-selection-breast-cancer>

1. Introduction

For this assignment I applied a Genetic Algorithm to **feature selection**: given 30 numerical features describing a tumor, I wanted to find which subset of them best predicts whether it's malignant or benign, using a GA to search for that subset instead of picking features by hand or checking every combination.

I went with this problem because with 30 binary features there are 2^{30} possible subsets - over a billion - which is far too many to brute-force, but small enough for a GA to search effectively by evolving progressively better subsets over generations. It also connects directly to the machine learning side of this course, not just the optimization side.

2. How a Genetic Algorithm works

A GA is a search technique loosely modeled on natural evolution. It keeps a **population** of candidate solutions, each represented as a **chromosome**. Every generation goes through the same cycle:

1. Score every chromosome with a **fitness function** - some measure of how good that candidate solution is.
2. **Select** fitter individuals more often to be parents (I used tournament selection: pick a few individuals at random, keep the best one).
3. **Crossover** combines two parents to produce offspring that mix traits from both.
4. **Mutation** makes small random changes to offspring, which keeps the population from getting stuck on the same handful of solutions.
5. The new generation (plus **elitism** - always keeping the current best individual around) replaces the old one, and the cycle repeats.

Over enough generations the population tends to drift toward better solutions without ever having to check every possibility.

3. The problem

Dataset: Breast Cancer Wisconsin (Diagnostic), which comes built into scikit-learn. 569 samples, 30 numerical features computed from digitized images of breast tissue samples (mean radius, mean texture, mean smoothness, worst concavity, and so on), labeled malignant or benign.

Goal: find a subset of these 30 features that gets at least as good classification accuracy as using all of them, ideally with noticeably fewer features - which would show that some of the 30 aren't actually pulling their weight.

4. How I applied the GA to this problem

Piece	What I used
Chromosome	Binary vector, length 30 (1 = feature selected, 0 = not)
Fitness	3-fold CV accuracy of a Random Forest (50 trees, kept small for speed), minus 0.0015 per feature used

Selection	Tournament selection, $k = 3$
Crossover	Uniform crossover, rate 0.8
Mutation	Bit-flip, rate 0.02 per gene
Elitism	Best individual kept every generation
Population / generations	20 / 30

I chose Random Forest as the classifier mostly for practical reasons: it doesn't need feature scaling (some of these 30 features are on very different scales, e.g. area vs. smoothness), and it trains fast enough to be retrained hundreds of times during the search without things taking forever. I also checked with someone who'd worked with this exact dataset before, and she'd found RF worked well on it too, which matched what I was already leaning toward.

On avoiding data leakage: the GA's fitness only ever sees the training set, evaluated through cross-validation. The test set (30% of the data) is never touched during the search - it only gets used once, right at the end, to get the final accuracy for both the baseline and whatever feature subset the GA ends up choosing. I didn't do this correctly on my first attempt (more on that below), so I want to be upfront that it took a fix.

I also added a small safeguard so a chromosome can never end up selecting zero features - if that would happen, one random feature gets forced on, since a zero-feature classifier isn't meaningful to evaluate.

5. Results

Single run

	Features used	Test accuracy
Baseline (all features, no GA)	30	0.9357
GA-selected features	8	0.9474

Features the GA picked (single run, 8 of 30): mean radius, mean area, radius error, smoothness error, compactness error, worst texture, worst area, worst concave points.

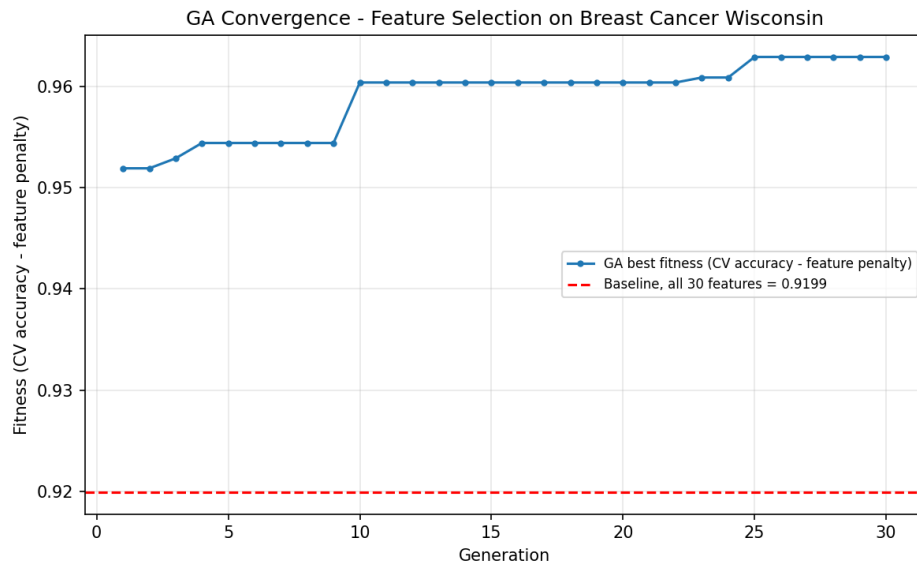


Figure 1. Best fitness per generation for a single run (seed 42). The red dashed line is the baseline computed with the same fitness formula, using all 30 features.

Across 5 different seeds

I didn't want to rely on a single lucky run, so I re-ran the GA with 5 different random seeds (same train/test split, only the GA's own randomness changes):

Metric	Value
Baseline test accuracy	0.9357
GA test accuracy (mean $\pm$ std)	0.9474 $\pm$ 0.0052
GA test accuracy (min - max)	0.9415 - 0.9532
Features used (mean $\pm$ std)	9.4 $\pm$ 2.0 out of 30
Seeds beating baseline	5 / 5

Seed	Features used	Test accuracy
1	10	0.9415
7	8	0.9532
21	13	0.9532
55	8	0.9474
99	8	0.9415

Every single seed found a smaller subset (8-13 features) that matched or beat the 30-feature baseline. The convergence plots show the GA settling into its best result somewhere between generation 10 and 25 in most runs.

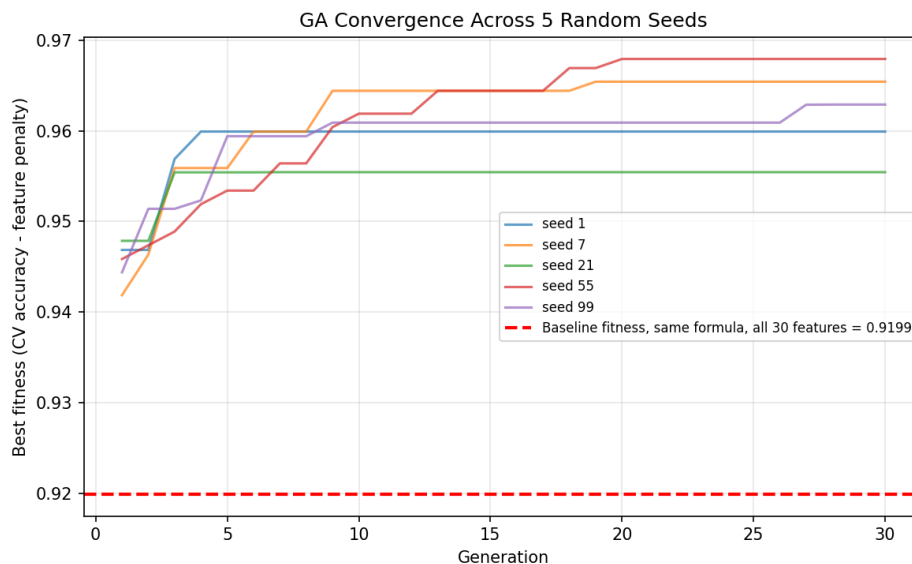


Figure 2. Best fitness per generation across 5 random seeds, with the same all-features baseline shown for reference.

6. Problems I ran into along the way

- **Data leakage in my first attempt.** Originally I computed fitness using one train/test split, which meant the GA could effectively "overfit" to that specific split without me realizing it. I fixed this by switching to cross-validation on the training data only, and keeping the test set completely separate until the very end.
- **My convergence plot was comparing two different things.** After I fixed the leakage issue, I noticed my baseline reference line on the plot was computed with different settings (more folds, more trees, no penalty) than what the GA itself was using internally. It still looked fine on the chart, but it wasn't really a fair comparison. I fixed it by computing the reference line with the exact same fitness formula the GA uses.
- **CV accuracy vs. test accuracy don't match exactly.** The baseline's cross-validation accuracy on the training data (0.9725) came out noticeably higher than its accuracy on the actual held-out test set (0.9357). That's normal for a dataset this size - just wanted to note it so it doesn't look like I missed something.
- **Runtime.** Retraining a classifier for every chromosome, every generation, is the slow part of this whole approach. I added a cache so repeated chromosomes (which happen a lot because of elitism) don't get re-evaluated, and used a smaller forest during the search itself.

7. Limitations

- This dataset is fairly "easy" to classify (the baseline is already at 93.6%), so the accuracy improvement from feature selection is real but modest - about 1 percentage point, even though it holds up across every seed I tested.
- I only tried Random Forest inside the fitness function. It would be worth checking whether the same feature subset still performs well with a different classifier (SVM, logistic regression), or whether the GA just found features that happen to suit RF specifically.
- A messier or higher-dimensional dataset would probably show a bigger benefit from feature selection than this one does.

8. Conclusion

This project applies a GA I wrote from scratch to feature selection on the Breast Cancer Wisconsin dataset, using a train/cross-validation/held-out-test setup that avoids data leakage. Across 5 independent runs, the GA consistently found smaller feature subsets (roughly a third of the original 30 features) that matched or slightly outperformed using every feature. It's not a dramatic accuracy jump, but it's a real and consistent one, and it demonstrates that a GA can search a combinatorial space this large efficiently without checking every possibility by hand.

Source code

GitHub repository: <https://github.com/osamaygh/ga-feature-selection-breast-cancer>

The repository contains the full code and a README with instructions to run it (python `main.py` for a single run, python `robustness.py` for the 5-seed check).

References

- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Wolberg, W. H., Street, W. N., & Mangasarian, O. L. Breast Cancer Wisconsin (Diagnostic) Data Set, UCI Machine Learning Repository.
- Week 3 lecture material, Computational Intelligence and Machine Learning.